

Apellidos:

Nombre:

Concurrencia y Paralelismo

Bloque I: Concurrencia

Julio 2024

1. Cola supermercado [1.5 puntos]

Implementar el esquema de la cola de un supermercado donde existe una única cola de clientes y múltiples cajeros. Cuando un cajero queda libre escoge el primer cliente de la cola. Los cajeros y los clientes no pueden hacer espera activa. Se pueden añadir todos los campos necesarios a los struct customer y super. No se pueden usar variables globales.

```
struct customer {
    pthread_cond_t c;
    ...
}

struct super {
    queue *q;
    pthread_mutex_t *m;
    pthread_cond_t *cajeros;
    ...
};

void cashier(struct super *s) {
    struct customer *c;
    while(true) {
        ...
        serve_customer(c);
    }
}

void customer(struct super *s, int num_items) {
    ...
    pay_and_leave();
}

struct cliente *peek(queue *q); // Devuelve, sin eliminarlo, el primer cliente en la cola
void remove(queue *q, struct customer *c); // Quita el cliente indicado de la cola
void insert(queue *q, struct customer *c); // Inserta un cliente al final de la cola
```

```
int pthread_mutex_init(pthread_mutex_t *m, pthread_mutex_attr_t *attr);
int pthread_mutex_lock(pthread_mutex_t *m);
int pthread_mutex_trylock(pthread_mutex_t *m);
int pthread_mutex_unlock(pthread_mutex_t *m);
int pthread_cond_init(pthread_cond_t *c, pthread_cond_attr_t *attr);
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mtx);
int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_broadcast(pthread_cond_t *cond);
```

Solución:

```
struct customer {
    pthread_cond_t c;
};

struct super {
    queue *q;
    pthread_mutex_t m;
};

void cashier(struct super *s) {
    struct cliente *c;
    while(true) {
        lock(&s->m);
        while((c = peek(&s->q)) == NULL)
            wait(&s->cajeros, &s->m);

        pthread_cond_signal(c->c);
        remove(&s->q, c);
        unlock(&s->m);
        serve_customer(c);
    }
}

void customer(struct super *s, int num_items) {
    struct customer *c = malloc(sizeof(struct customer));
    pthread_cond_init(&c->c, NULL);

    pthread_mutex_lock(&s->m);
    insert(s->q, c);
    pthread_cond_signal(s->cajeros);
    pthread_cond_wait(&c->c, &s->m);
    pthread_mutex_unlock(&s->m);
    pay_and_leave();
}
```

2. Mutex con prioridades [2 puntos]

Implemente, utilizando los mutex de la librería pthread, un tipo de mutex donde se pueden hacer bloqueos con prioridad alta o baja. Cuando un mutex se libera, solo podrá ser bloqueado por un thread con prioridad baja si no hay ningún thread con prioridad alta esperando.

```
typedef {
    pthread_mutex_t m;
    pthread_cond_t c;
    ...
} prio_mutex;

void high_prio_lock(prio_mutex *m) {
    ...
}

void low_prio_lock(prio_mutex *m) {
    ...
}

void prio_unlock(prio_mutex *m) {
    ...
}
```

Implemente las operaciones high_prio_lock, low_prio_lock y prio_unlock.

Solución:

```
typedef {
    pthread_mutex_t m;
    pthread_cond_t c;
    int waiting_high_prio;
    int locked;
} prio_mutex;

void high_prio_lock(prio_mutex *m) {
    pthread_mutex_lock(&m->m);
    while(m->locked) {
        m->waiting_high_prio++;
        pthread_cond_wait(&m->c, &m->m);
        m->waiting_high_prio--;
    }
    m->locked = 1;
    pthread_mutex_unlock(&m->m);
}

void low_prio_lock(prio_mutex *m) {
    pthread_mutex_lock(&m->m);
    while(m->locked || m->waiting_high_prio > 0)
        pthread_cond_wait(&m->c, &m->m);

    m->locked=1;
    pthread_mutex_unlock(&m->m);
}

void prio_unlock(prio_mutex *m) {
    pthread_mutex_lock(&m->m);
    m->locked = 0;
    pthread_cond_broadcast(&m->c);
    pthread_mutex_unlock(&m->m);
}
```

3. Servidor de datos etiquetados [1.5 puntos]

Escriba un módulo que permita crear procesos servidor con la siguiente interfaz:

- `start()`, que arranca un proceso servidor y devuelve su PID.
- `put(S, Data, Tags)`, donde `S` es el PID de un proceso servidor, `Data` un dato arbitrario, y `Tags` una lista de etiquetas que describen el dato. El dato debe quedar almacenado en el servidor.
- `get(S, Tags)`, donde `S` es el PID de un proceso servidor, y `Tags` una lista de etiquetas. La función debe devolver una lista con todos los datos almacenados que tienen todas las etiquetas que están en `Tags`.

```
-module(store).
-export([start/0, get/2, put/3]).

start() ->
    spawn(?MODULE, init, []).
put(S, Data, Tags) ->
    ...
get(S, Tags) ->
    ...

init() ->
    ...
```

Por ejemplo:

```
1> S = store:start().
<0.10.0>
2> store:put(S, one, [green, large]).
ok
3> store:put(S, two, [small]).
ok
4> store:get(S, []).
[one, two]
5> store:get(S, [green]).
[one]
6> store:get(S, [red]).
[]
```

Solución:

```
-module(store).
-export([start/0, get/2, put/3]).

start() ->
    spawn(?MODULE, init, []).
put(S, Data, Tags) ->
    S ! {put, Data, Tags},
    ok.
get(S, Tags) ->
    S ! {get, Tags, self()},
    receive
        {get_reply, L} -> L
    end.

init() ->
    loop([]).

loop(L) ->
    receive
        {put, Data, Tags} ->
            loop([Data, Tags | L]);
        {get, Tags, From} ->
            Res = [Data || {Data, DTags} <- L, lists:all(fun(T) -> lists:member(T, DTags) end, Tags)],
            From ! {get_reply, Res},
            loop(L)
    end.
```

EXAMEN DE CONCURRENCIA Y PARALELISMO

Bloque II: Paralelismo

APELLIDOS: _____ NOMBRE: _____

Convocatoria extraordinaria

12 de julio de 2024

AVISO: No mezcléis en el mismo folio preguntas del bloque de concurrencia y del bloque de paralelismo. Las respuestas a cada bloque se entregan por separado.

- [2,5p] 1. **Conceptos de paralelismo.** En un hospital se dispone de un software que permite analizar radiografías de diferentes partes del cuerpo trabajando pixel a pixel. Se quiere desarrollar un programa paralelo que permita acelerar, usando varios procesos, el análisis de una radiografía de última generación.

A causa del hardware disponible en el hospital solo uno de los procesos tendrá acceso a disco para leer la imagen (con un tiempo constante de un minuto para leer una radiografía) y escribir el resultado del análisis (en un tiempo despreciable). Este proceso también puede realizar parte del análisis. Los procesos solo pueden empezar el análisis cuando todos tienen ya la imagen en su memoria. El análisis de una radiografía por un único proceso tarda 19 minutos y se realiza en cada pixel de forma independiente, pero sólo suponen carga de trabajo aquellos pixels que representan hueso dentro de la imagen. Responde RAZONADAMENTE a las siguientes cuestiones:

- (a) Indica qué tipo de descomposición y asignación de tareas usarías. [0,5p]
- (b) ¿Cuál es la máxima aceleración teórica que se puede conseguir si consiguiésemos reducir el tiempo de análisis al máximo? [0,5p]
- (c) En una primera versión todos los procesos necesitan la radiografía completa aunque vayan a hacer solo un análisis parcial. Un envío de una imagen desde el Proceso 0 a cualquiera de los otros tarda siempre 10 segundos. El reparto de trabajo no es perfecto, lo que provoca que siempre uno de los procesos se ocupe de la mitad de la carga de trabajo dentro de la fase de análisis, quedando la otra mitad a repartir entre el resto de procesos. Calcula las siguientes métricas para este código en una ejecución con un total de 4 procesos: [1,0p]
 - i. Tiempo paralelo
 - ii. Speedup
 - iii. Eficiencia
 - iv. Coste
 - v. Sobrecarga
- (d) Asume que ahora queremos hacer una aproximación paralela diferente, donde en vez de abordar la paralelización de cada radiografía por separado se asume que en el hospital se deben analizar varias radiografías en un corto periodo de tiempo, y cada radiografía puede requerir un tiempo distinto, según la zona del cuerpo. En este caso se va a instalar un servidor que se dedica exclusivamente a leer las radiografías, distribuirlas al resto de servidores, y recopilar los resultados. Indica cómo cambiarías la descomposición y la asignación de tareas en este caso. [0,5p]

- [2,5p] 2. **Diseño de algoritmos paralelos.** Se quiere paralelizar el siguiente código que trabaja sobre una matriz A de dos dimensiones (M filas y N columnas). El resultado de la última fila y la última columna es siempre 0:

```
float *A = (float *)malloc(sizeof(float) * M * N);
float *res = (float *)malloc(sizeof(float) * M * N);
float aux;
inicializa(A);

for(int i = 0; i < M-1; i++){
    for(int j = 1; j < N-1; j++){
        aux = A[i*N+j]*A[(i+1)*N+j+1]
              -A[i*N+j+1]*A[(i+1)*N+j]);
        res[i*N+j] = aux;
    }
    res[(i+1)*N-1] = 0;
}
for(int j = 1; j < N; j++){
    res[(M-1)*N+j] = 0;
}

escribeResultado(res);
```

Solo uno de los procesos tiene acceso a disco y pantalla. Por tanto, este será el único proceso que puede ejecutar las funciones `inicializa` y `escribeResultado`. Además, M es siempre múltiplo del número de procesos. Contesta DE FORMA RAZONADA las siguientes preguntas:

- (a) ¿Qué tipo de descomposición y asignación de tareas usarías? ¿A qué datos debe acceder cada proceso? [0,75p]
- (b) Escribe un código MPI que permita ejecutar de forma paralela este programa, usando colectivas siempre que se pueda (ver la tabla con la sintaxis de las rutinas MPI). [1,0p]
- (c) Supón un sistema donde un proceso con rank X solo puede comunicarse con los procesos con ranks $X - 1$ y $X + 1$. Implementa una función ad-hoc que, usando exclusivamente comunicaciones punto a punto, haga todas las comunicaciones necesarias para la distribución de la matriz A de acuerdo a la distribución de datos que has elegido en el apartado (a). Asume que inicialmente la matriz A siempre estará en el Proceso 0. ¿Habría que modificar el tamaño de alguno de los arrays empleados en el apartado (b)? [0,75p]

```
int MPI_Send(void *buf, int count, MPI_Datatype datatype,
             int dest, int tag, MPI_Comm comm)
int MPI_Recv(void *buf, int count, MPI_Datatype datatype,
             int source, int tag, MPI_Comm comm, MPI_Status *status)
int MPI_Bcast(void *buffer, int count, MPI_Datatype datatype,
             int root, MPI_Comm comm)
int MPI_Scatter(void *sendbuf, int sendcnt, MPI_Datatype sendtype,
               void *recvbuf, int recvcnt, MPI_Datatype recvtype,
               int root, MPI_Comm comm)
int MPI_Gather(void *sendbuf, int sendcnt, MPI_Datatype sendtype,
               void *recvbuf, int recvcount, MPI_Datatype recvtype,
               int root, MPI_Comm comm)
int MPI_Reduce(void *sendbuf, void *recvbuf, int count,
               MPI_Datatype datatype, MPI_Op op,
               int root, MPI_Comm comm)
```

SOLUCIÓN

1. (a) Aplicaríamos una descomposición de dominio donde el análisis de cada pixel sería una tarea, ya que son independientes. La asignación sería estática porque se sabe el tipo de imagen que hay al inicio de la ejecución. Para aliviar el problema del desbalanceo de carga por culpa de que no hay que trabajar con algunos pixels aplicaría una distribución cíclica, ya que los pixels que representan masa ósea probablemente estén rodeados de otros también con masa ósea (y viceversa).
 - (b) Siguiendo la ley de Amdhal, el tiempo paralelo en el mejor caso sería el de la parte que no se puede paralelizar (lectura y escritura), ya que el tiempo de análisis se reduciría hasta ser casi despreciable. Como la escritura no consume tiempo, el tiempo paralelo es, en el mejor caso, el minuto de lectura de la imagen. El tiempo secuencial este minuto más los 19 minutos de análisis. Por tanto, la aceleración máxima sería: $\frac{20}{1} = 20$
 - (c)
 - i. La parte de lectura no se puede paralelizar, así que en toda ejecución tendremos 60 segundos de lectura. Además hay que sumarle 10 segundos por envío de la imagen a cada proceso (30 segundos de comunicaciones, en total). Por último, el tiempo de análisis es, como mínimo, la mitad del tiempo de análisis secuencial (9 minutos y medio o 570 segundos), porque un proceso ya es el que necesita. Esto hace que el tiempo paralelo sea: $60 + 3 * 10 + 570 = 660$ segundos.
 - ii. La aceleración consiste en dividir el tiempo secuencial (20 minutos) entre el tiempo paralelo: $\frac{1200}{660} = 1,82$
 - iii. La eficiencia consiste en dividir el speedup entre el número de procesos: $\frac{1,82}{4} = 0,45$
 - iv. El coste es el tiempo paralelo por el número de procesos: $4 * 660 = 2640$
 - v. La sobrecarga es restar el coste menos el tiempo secuencial: $2640 - 1200 = 1440$
 - (d) En este caso seguiría siendo una descomposición de dominio, pero cada tarea sería el análisis de una radiografía distinta. En cuanto a la asignación se haría una dinámica maestro-esclavo, que ayudará a aliviar la diferente carga de trabajo de diferentes imágenes.
2. (a) Se aplica una descomposición de dominio con una asignación estática bloque por filas. Se podría también hacer por columnas o por bloques 2D pero complicaría mucho la implementación. Hay que tener en cuenta que cada proceso (a excepción del último) necesita también la primera fila del siguiente bloque para poder calcular el resultado.


```

(b) float *A, *res, *myA, *myres;
    float aux;
    MPI_Status status;
    myA = (float *) malloc(sizeof(float) * (M/numP+1) * N);
    myres = (float *) malloc(sizeof(float) * M/numP * N);

    if(!rank) {
        A = (float *) malloc(sizeof(float) * M * N);
        res = (float *) malloc(sizeof(float) * M * N);
        inicializa(A);
    }

    MPI_Scatter(A, M/numP * N, MPI_FLOAT, myA, M/numP * N,
        MPI_FLOAT, 0, MPI_COMM_WORLD);

    if(rank > 0)
        MPI_Send(myA, N, MPI_FLOAT, rank-1, 0, MPI_COMM_WORLD);

    int numFilas = M/numP-1;
    if(rank < numP-1){
        MPI_Recv(&myA[M/numP*N], N, MPI_FLOAT, rank+1, 0,
            MPI_COMM_WORLD, &status);
        numFilas = M/numP;
    }

    for(int i = 0; i < numFilas; i++)
        for(int j = 1; j < N-1; j++){
            aux = myA[i*N+j]*myA[(i+1)*N+j+1]-myA[i*N+j+1]*myA[(i+1)*N+j];
            myres[i*N+j] = aux;
        }
        myres[(i+1)*N-1] = 0;

    MPI_Gather(myres, M/numP*N, MPI_FLOAT, res, M/numP*N,
        MPI_FLOAT, 0, MPI_COMM_WORLD);

    if(!rank) {
        for(int j = 1; j < N; j++){
            res[(M-1)*N+j] = 0;
            escribeResultado(det);
        }
    }

```

- (c) En este caso el Proceso 0 empezará mandando al Proceso 1 todas las filas que necesita tanto él como los procesos siguientes. El Proceso 1 al 2 las que necesita el 2 y los siguientes, etc. Ten en cuenta que, a diferencia del apartado b, el array *myA* debe ser diferente en cada proceso para poder almacenar los datos tanto de ese proceso como de los siguientes.

```
MPI_Status status;
int bloqueFilas = M/numP*N;

if(!rank)
    MPI_Send(&A[bloqueFilas], bloqueFilas*(numP-1), MPI_FLOAT, 1, 0,
             MPI_COMM_WORLD);

if(rank == numP-1)
    MPI_Recv(myA, bloqueFilas, MPI_FLOAT, numP-2, 0,
             MPI_COMM_WORLD, &status);

if(rank > 0 && rank < numP-1){
    MPI_Recv(myA, bloqueFilas*(numP-rank), MPI_FLOAT, rank-1, 0,
             MPI_COMM_WORLD, &status);
    MPI_Send(&myA[bloqueFilas], bloqueFilas*(numP-rank-1), MPI_FLOAT,
             rank+1, 0, MPI_COMM_WORLD);
}
```